

Laboratorio di Sistemi Operativi A.A. 2025-26

Progetto: rete di nodi sensore con aggregatore centrale

Nome del gruppo: SistemTeam

Email del referente: pierfrances.musolesi@studio.unibo.it

Componenti del gruppo

Nome	Cognome	Matricola	Area di lavoro
Pierfrancesco	Musolesi	0001173656	B – Rete e console dell'aggregatore
Irene	Benevieri	0001175851	A – Protocollo e stato dell'aggregatore
Bianca	Garagnani	0001193380	C – Avvio e console del nodo sensore
Francesco	Battistini	0001160667	D – Comunicazione fra nodi e download

Contenuto del documento

1. Introduzione
2. Architettura generale
3. Le componenti nel dettaglio
4. Concorrenza: risorse condivise e meccanismi di sincronizzazione
5. Il modello client-server: connessioni, chiusure e guasti
6. Problemi incontrati, soluzioni e alternative
7. Suddivisione del lavoro
8. Strumenti utilizzati
9. Compilazione e uso, con esempi

1. Introduzione

Il progetto realizza una piccola rete distribuita in Java in cui più **nodi sensore** condividono le proprie rilevazioni (temperature, pressioni, livelli di CO2) attraverso un nodo centrale, l'**aggregatore**.

Ogni nodo sensore conserva le proprie rilevazioni in una cartella di file di testo. L'aggregatore invece non conserva nessuna rilevazione: tiene solo l'elenco di **chi possiede cosa** e indirizza i nodi l'uno verso l'altro. Quando un nodo scarica una rilevazione, il contenuto passa direttamente dal nodo che la possiede a quello che la chiede, senza transitare dall'aggregatore.

Il progetto è formato da due programmi, avviati da terminale: **Aggregator**, il nodo centrale, e **Client**, il nodo sensore, di cui si possono avviare quante copie si vuole. La comunicazione avviene tramite socket TCP, e la concorrenza è gestita esclusivamente con i costrutti: `synchronized`, `wait()` e `notifyAll()`.

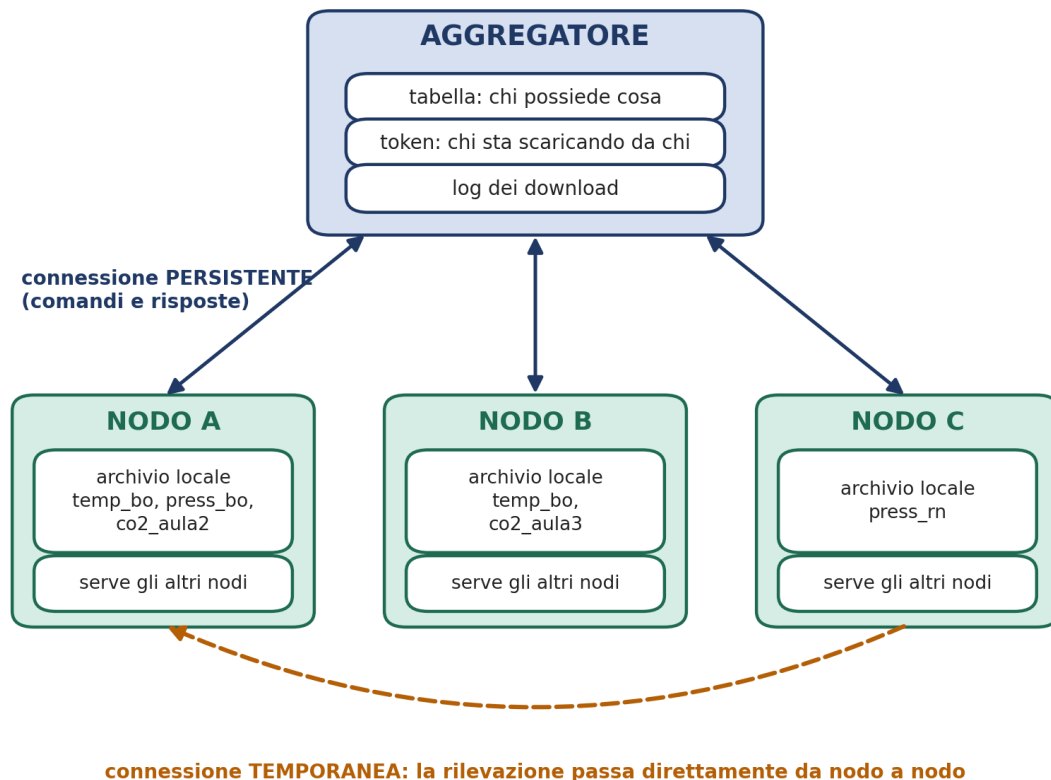
1.1 Alcuni termini usati nel documento

Termine	Significato
Rilevazione	Un dato identificato da un nome univoco, per esempio <code>temp_bo</code> . Sul disco del nodo è il file <code>temp_bo.txt</code> .
Nodo sensore (peer)	Una copia del programma <code>Client</code> . Alla registrazione l'aggregatore gli assegna un nome: <code>peer0</code> , <code>peer1</code> , <code>peer2</code> ...
Tabella delle rilevazioni	L'elenco, tenuto dall'aggregatore, di quali nodi possiedono ciascuna rilevazione.
Token	Un permesso, rilasciato dall'aggregatore, per scaricare da un certo nodo. Esiste un token per ogni nodo: chi lo trova occupato aspetta.
Thread	Un flusso di esecuzione indipendente dentro un programma. Più thread lavorano contemporaneamente.
Risorsa condivisa	Un dato usato contemporaneamente da più thread, che va quindi protetto.
Monitor	Il "lucchetto" che ogni oggetto Java possiede. Un metodo <code>synchronized</code> lo prende all'ingresso e lo lascia all'uscita, così un solo thread alla volta può eseguirlo.

2. Architettura generale

2.1 Le componenti e le connessioni

La Figura 1 mostra una rete con l'aggregatore e tre nodi sensore, con le rilevazioni di esempio fornite nel repository (cartella data/).



L'aggregatore custodisce tre informazioni: la **tabella** di chi possiede cosa, i **token** che regolano chi sta scaricando da chi, e il **registro dei download** mostrato dal comando log. Ogni nodo sensore ha il proprio **archivio locale** e una parte che resta in ascolto per **servire gli altri nodi**.

Fra le componenti esistono due tipi di connessione, con scopi diversi:

- **Connessione persistente** fra ogni nodo e l'aggregatore. Si apre all'avvio del nodo e si chiude solo con quit. Su di essa viaggiano i comandi: registrazione, richiesta degli elenchi, richiesta di un nodo da cui scaricare, notifiche di aggiunta e rimozione. Tenerla sempre aperta ha un vantaggio preciso: se un nodo cade, la connessione si interrompe e l'aggregatore se ne accorge subito.
- **Connessione temporanea** fra due nodi sensore. Si apre solo per trasferire una rilevazione e si chiude subito dopo. L'aggregatore non vede mai passare il contenuto.

2.2 Come si parlano: il protocollo

Aggregatore e nodi sono programmi separati, spesso su computer diversi: non possono chiamarsi metodi a vicenda, possono solo scambiarsi byte su un socket. L'accordo su cosa significano quei byte sta in un unico file condiviso dalle due applicazioni, `Protocol.java`. Le regole sono poche:

- i messaggi sono **testo**, una riga per messaggio;
- a ogni richiesta corrisponde **esattamente una risposta**, che comincia con OK oppure con ERR seguito da un codice (NOTFOUND, BADREQUEST, NOTREGISTERED, BADTOKEN). L'aggregatore non invia mai nulla di sua iniziativa;
- quando la risposta è un **elenco**, la prima riga dice quante righe seguono: OK 3 e poi tre righe;
- quando la risposta è il **contenuto** di una rilevazione, una riga LENGTH 37 annuncia quanti byte seguono.

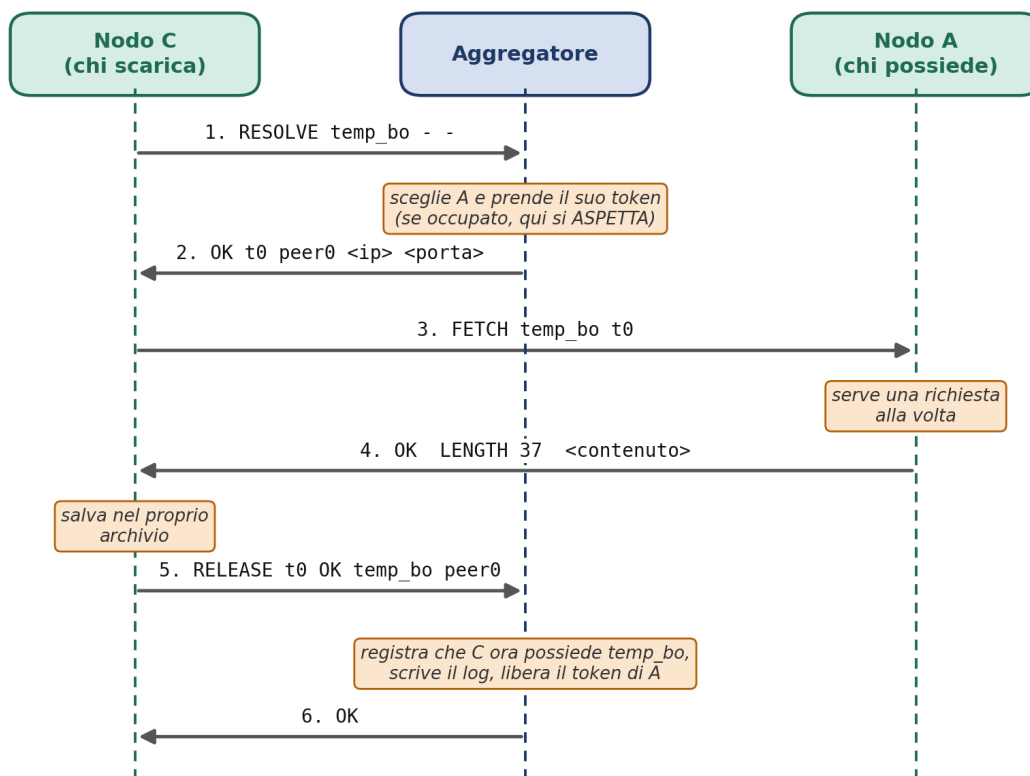
Abbiamo scelto un protocollo di testo perché si può provare a mano: con il programma nc (netcat, una connessione TCP) ci si collega all'aggregatore e si digitano i messaggi uno per uno, vedendo le risposte. Durante lo sviluppo lo abbiamo usato per verificare l'aggregatore prima che il nodo sensore esistesse.

Messaggio	A cosa serve	Risposta
REGISTER <porta> <n> più n righe con i nomi	Registrarsi all'avvio, dichiarando porta e rilevazioni	OK <peerId>
LIST <nome oppure ->	Comando listdata remote	OK <n> più n righe
PEERS	Comando listdata peers	OK <n> più n righe
ADDED <nome> REMOVED <nome>	Notificare add e remove	OK
RESOLVE <nome> <token> <esclusi>	Chiedere da quale nodo scaricare	OK <token> <peer> <host> <porta> oppure ERR NOTFOUND
RESOLVE_AT <nome> <peer> <token>	Download da un nodo scelto dall'utente	Come sopra
RELEASE <token> <OK FAIL> <nome> <sorgente>	Chiudere un download e liberare il token	OK oppure ERR BADTOKEN
DISCONNECT	Comando quit del nodo	OK
FETCH <nome> <token> (fra due nodi)	Scaricare il contenuto	OK, LENGTH <n>, n byte oppure ERR NOTFOUND

Il segno - indica un argomento assente: i campi di un messaggio sono posizionali, quindi un campo vuoto sposterebbe tutti quelli successivi.

2.3 Esempio: il percorso completo di un download

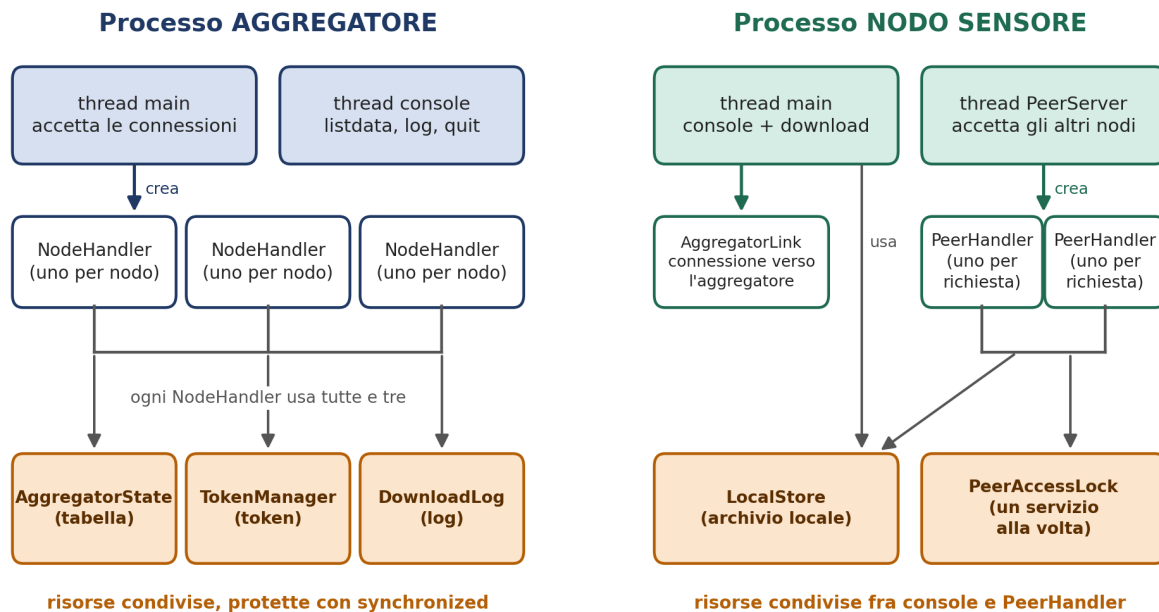
La Figura 2 segue un download dall'inizio alla fine. Il nodo C vuole la rilevazione temp_bo, che possiede il nodo A (peer0).



1. C chiede all'aggregatore da chi scaricare temp_bo con una RESOLVE.
2. L'aggregatore consulta la tabella, sceglie A e prende per C il **token di A**. Se il token di A è già in uso da un altro nodo, la richiesta di C resta in attesa. Poi risponde con il token e con l'indirizzo di A.
3. C si collega direttamente ad A e chiede il contenuto con una FETCH.
4. A risponde con OK, la lunghezza e i byte della rilevazione. A serve una richiesta alla volta: se ne arrivano altre, aspettano il loro turno.
5. C salva la rilevazione nella propria cartella e lo comunica all'aggregatore con una RELEASE.
6. L'aggregatore registra che ora anche C possiede temp_bo, scrive una riga nel log e libera il token di A, svegliando chi lo stava aspettando.

3. Le componenti nel dettaglio

I due programmi sono formati da quindici classi in tutto, ciascuna con un compito preciso. La Figura 3 mostra i thread di ciascun processo e le risorse condivise che usano (in arancione).



3.1 Il nodo aggregatore

L'aggregatore accetta le connessioni dei nodi, mantiene la tabella delle rilevazioni, assegna i token, registra i download e offre una console all'utente. Lavora con tre tipi di thread:

- il **thread main** resta fermo ad accettare connessioni. Per ognuna crea un **NodeHandler**, lo avvia e torna subito ad accettare: fra una connessione e l'altra non fa nulla di lento, altrimenti i nodi che si collegano nel frattempo resterebbero in coda;
- il **thread della console** legge i comandi da tastiera. È separato perché la lettura da tastiera è bloccante: se la facesse il main, nessun nodo potrebbe collegarsi mentre l'aggregatore aspetta che qualcuno digiti, e le specifiche chiedono che la sessione interattiva non interrompa il funzionamento nella rete;
- un **thread NodeHandler per ogni nodo** collegato. Legge una richiesta, la esegue, risponde e ricomincia. Così un nodo lento, o in attesa di un token, non rallenta tutti gli altri.

Classe	Compito	Metodi principali
Aggregator	Avvio, creazione degli oggetti condivisi, accettazione delle connessioni, chiusura ordinata	avvia(), accettaConnessioni(), arresta()
NodeHandler	Un thread per nodo: esegue i comandi ricevuti e fa pulizia quando il nodo se ne va	register(), resolve(), release(), pulizia()
AggregatorConsole	Console dell'aggregatore	listdata(), log()
AggregatorState	Tabella delle rilevazioni e anagrafica dei nodi (Meccanismo 1)	completaRegistrazione(), risolvi(), elencoRisorse()
TokenManager	Token di accesso ai nodi (Meccanismo 3)	acquisisci(), rilascia(), rilasciaTuttiDi()
DownloadLog	Registro dei download per il comando log	aggiungi(), righe()

Un principio guida tutte le classi che custodiscono dati (AggregatorState, TokenManager, DownloadLog): **non chiamano mai un'altra classe sincronizzata e non scrivono mai sulla rete**. Lavorano solo in memoria e ritornano. È il NodeHandler che le chiama una alla volta, in sequenza. Questo garantisce che un thread non tenga mai due lucchetti contemporaneamente, che è la condizione per evitare i blocchi reciproci (deadlock).

3.2 Il nodo sensore

Il nodo sensore gestisce l'archivio locale, la console dell'utente, il servizio verso gli altri nodi e i download. Anche qui i thread sono tre tipi:

- il **thread main** esegue la console. Anche i download girano su questo thread: mentre un download è in corso la console di quel nodo è occupata, ma il nodo continua a funzionare nella rete;
- il **thread PeerServer** accetta le connessioni degli altri nodi. Si mette in ascolto sulla porta 0, cioè lascia scegliere al sistema operativo una porta libera: così più nodi possono girare sulla stessa macchina senza scontrarsi. La porta scelta viene poi comunicata all'aggregatore;
- un **thread PeerHandler** per ogni richiesta ricevuta da un altro nodo.

Classe	Compito	Metodi principali
Client	Avvio del nodo: argomenti, archivio, connessione, PeerServer, registrazione, console	main()
ClientConsole	Comandi dell'utente e formati di output	listdataRemote(), add(), remove(), quit()
AggregatorLink	L'unica connessione verso l'aggregatore	invia(), inviaConAttesaIndefinita()
LocalStore	Archivio su disco: la cartella di file .txt	elenco(), leggi(), aggiungi(), rimuovi()
PeerServer	Accetta le connessioni degli altri nodi	run(), getPorta()
PeerHandler	Serve una singola richiesta FETCH	run()
PeerAccessLock	Fa servire una richiesta alla volta (Meccanismo 2)	acquisisci(), rilascia()
DownloadManager	Download con i tentativi ripetuti	download(nome), download(peer, nome)

3.3 La classe condivisa Protocol

Protocol.java è usata da entrambi i programmi. Contiene i nomi dei messaggi e i metodi per leggere e scrivere sul socket: `readLine()` legge una riga, `readFully()` legge un contenuto di lunghezza nota, `writeLine()` e `writePayload()` scrivono, `parseInt()` e `split()` interpretano i campi.

È usata da moltissimi thread insieme ma **non ha bisogno di alcuna sincronizzazione**, perché non ha stato: solo costanti e metodi che lavorano sui propri parametri. Ogni thread che la chiama lavora sulle proprie variabili, e non c'è nulla di condiviso da proteggere.

4. Concorrenza: risorse condivise e meccanismi di sincronizzazione

Una risorsa è condivisa quando più thread la usano contemporaneamente. In questo caso va protetta, altrimenti due thread che la modificano insieme possono perdere i cambiamenti l'uno dell'altro. La tabella elenca tutte le risorse condivise del progetto.

Risorsa	Chi la usa contemporaneamente	Come è protetta
Tabella delle rilevazioni (AggregatorState)	Tutti i NodeHandler e la console	Metodi synchronized
Token (TokenManager)	Tutti i NodeHandler	synchronized, wait() e notifyAll()
Registro dei download (DownloadLog)	I NodeHandler scrivono, la console legge	Metodi synchronized
Lista dei nodi collegati (Aggregator)	Accettazione, handler che terminano, quit	synchronized(handlers)
Archivio locale (LocalStore)	Console del nodo e PeerHandler	Metodi synchronized
Nodo che serve (PeerAccessLock)	Tutti i PeerHandler dello stesso nodo	synchronized, wait() e notifyAll()
Connessione all'aggregatore (AggregatorLink)	Console e download	synchronized sull'intero scambio

4.1 Meccanismo 1: la tabella dell'aggregatore

Le specifiche chiedono che, mentre la tabella viene aggiornata, **ogni altra richiesta resti in attesa**, anche quelle che vogliono solo leggerla. La soluzione è semplice: tutti i metodi pubblici di AggregatorState sono synchronized sullo stesso oggetto. In Java un thread dentro un metodo synchronized tiene il lucchetto dell'oggetto, e nessun altro thread può entrare in nessun altro metodo synchronized dello stesso oggetto finché non ha finito.

Tre accorgimenti rendono questo meccanismo sicuro ed efficiente:

- **La registrazione è un'operazione unica.** Il NodeHandler legge tutte le righe della REGISTER prima di toccare la tabella, e solo alla fine chiama completaRegistrazione(). Così nessuno vede mai una registrazione a metà, e un nodo lento a inviare le righe non fa aspettare nessuno (vedi AggregatorState.java, riga 89).

- **Le letture restituiscono copie.** Gli elenchi escono dal metodo come liste nuove, quindi la stampa avviene fuori dal lucchetto e nessun nodo aspetta mentre l'utente legge.
- **I metodi sono brevi.** Lavorano solo in memoria e durano pochi microsecondi, quindi l'attesa imposta dalla specifica non rallenta la rete.

4.2 Meccanismo 2: un nodo serve una richiesta alla volta

Le specifiche chiedono che un nodo sensore serva una sola richiesta alla volta, e che le altre **restino in attesa senza mai terminare con un errore**. PeerAccessLock realizza questa regola con un indicatore "occupato" (vedi PeerAccessLock.java, righe 17 e 37):

```
acquisisci(): while (occupato) { wait(); }
               occupato = true;

rilascia():   occupato = false;
               notifyAll();
```

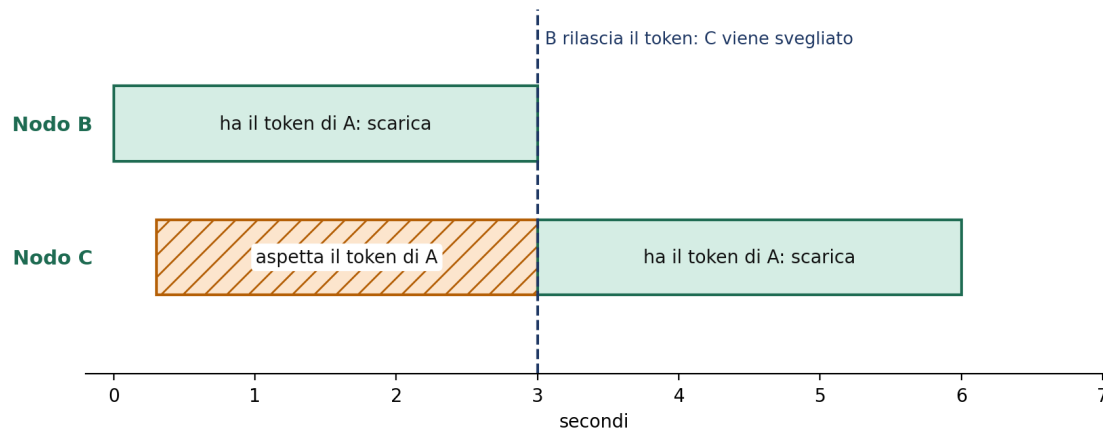
Chi trova il nodo occupato si addormenta con `wait()`, che lascia libero il lucchetto; chi finisce mette l'indicatore a falso e sveglia tutti con `notifyAll()`. Si usa **while e non if**: un thread appena svegliato deve ricontrollare, perché un altro potrebbe essere passato prima di lui. Il PeerHandler prende il permesso prima di servire e lo rilascia in un blocco `finally`, così anche un trasferimento fallito libera il nodo (vedi PeerHandler.java, righe 84 e 151).

Il permesso lo prende chi serve, non chi chiede. Se lo prendesse chi chiede, due nodi che scaricano uno dall'altro nello stesso momento resterebbero bloccati per sempre: A occupato a chiedere a B non potrebbe servire B, e B occupato a chiedere ad A non potrebbe servire A. Prendendolo dal lato di chi serve, un nodo che sta scaricando resta comunque libero di servire gli altri.

Esempio reale. Abbiamo avviato il nodo A con un ritardo di 3 secondi per ogni trasferimento e inviato due richieste contemporanee: la prima ha ricevuto risposta dopo 3,0 secondi, la seconda dopo 5,8 secondi, senza errori. Nel terminale di A compariva il messaggio in attesa... per la seconda richiesta.

4.3 Meccanismo 3: il token

Le specifiche dicono che il comando `download` richiede all'aggregatore un token di accesso e lo rilascia alla fine: il token **garantisce la mutua esclusione nell'accesso al nodo da cui si scarica**, vale per ciascun nodo e non per l'intera rete. Se due nodi vogliono scaricare dallo stesso nodo, uno procede e l'altro aspetta; download da nodi diversi procedono invece in parallelo.



TokenManager tiene, per ogni nodo, chi sta usando il suo token. Chi lo chiede quando è occupato si addormenta, esattamente come nel Meccanismo 2 (vedi TokenManager.java, righe 49-55):

```
acquisisci(sorgente): while (token di sorgente occupato) { wait(); }
                      assegna il token al richiedente

rilascia(token):      libera il token della sorgente
                      notifyAll();
```

Qui l'uso di notifyAll() e non di notify() è indispensabile: sullo stesso oggetto possono dormire thread che aspettano il token di nodi **diversi**. notify() ne sveglierebbe uno a caso, magari uno che aspetta un altro nodo e che torna subito a dormire, mentre quello che poteva procedere resterebbe addormentato per sempre.

Perché TokenManager è separato dalla tabella. Chi aspetta un token può restare fermo anche molti secondi. Se l'attesa avvenisse tenendo il lucchetto della tabella, tutta la rete si fermerebbe: nessuno potrebbe nemmeno fare listdata. Per questo il NodeHandler, gestendo una RESOLVE, prima sceglie il nodo sotto il lucchetto della tabella, **poi lo rilascia**, e solo dopo chiede il token (vedi NodeHandler.java, metodo resolve(), riga 218).

Esempio reale. Con i nodi B e C che chiedono lo stesso file ad A nello stesso momento, l'aggregatore stampa:

```
peer1 ottiene t1 su peer0
peer2 ottiene t2 su peer0 dopo 2.768 s di attesa
```

Due meccanismi insieme. Il token e il Meccanismo 2 sembrano fare la stessa cosa, ma lavorano a livelli diversi. Il token è un coordinamento generale, deciso dall'aggregatore; il lucchetto del nodo è una garanzia locale, con cui il nodo si protegge da solo, anche da chi si collega senza passare dall'aggregatore. I due permessi si prendono sempre nello stesso ordine (prima il token, poi il lucchetto del nodo), e questo esclude che si blocchino a vicenda.

4.4 Le regole valide in tutto il progetto

- **Un lucchetto alla volta.** Nessun metodo `synchronized` ne chiama un altro di un oggetto diverso: è la condizione per evitare i deadlock visti a lezione.
- **Mai un lucchetto durante un'operazione di rete**, perché l'attesa di un altro computer può durare indefinitamente. C'è una sola eccezione, voluta e motivata: `AggregatorLink` (vedi paragrafo 5.2).
- **Sempre `while` prima di `wait()`, sempre `notifyAll()`.**
- **Thread creati con `implements Runnable`.**
- **Solo `synchronized`, `wait()` e `notifyAll()`.**

5. Il modello client-server: connessioni, chiusure e guasti

5.1 Come si aprono le connessioni

L'**aggregatore** apre il proprio `ServerSocket` sulla porta indicata una sola volta, fuori dal ciclo di accettazione: se la porta è occupata il programma si ferma subito con un messaggio chiaro. Poi accetta le connessioni e affida ciascuna a un nuovo thread.

Il **nodo sensore** si avvia in un ordine preciso (vedi `Client.java`):

1. legge gli argomenti e apre l'archivio locale;
2. si collega all'aggregatore, con un messaggio d'errore se non è raggiungibile;
3. avvia il proprio `PeerServer`, ottenendo dal sistema operativo una porta libera;
4. si registra con `REGISTER`, dichiarando quella porta e i nomi delle proprie rilevazioni;
5. avvia la console.

Il `PeerServer` parte prima della registrazione perché la registrazione deve contenere la sua porta. L'indirizzo del nodo, invece, non lo dichiara il nodo: l'aggregatore lo ricava dalla connessione stessa. Un nodo potrebbe dichiarare un indirizzo sbagliato, per esempio se ha più schede di rete; quello da cui la connessione arriva davvero è sempre corretto.

5.2 Come si mantengono

La connessione fra nodo e aggregatore resta aperta per tutta la vita del nodo e funziona a domanda e risposta. Il nodo la usa attraverso `AggregatorLink`, che è **l'unico punto del progetto in cui si tiene un lucchetto durante un'operazione di rete**. È necessario: se due thread del nodo potessero usare la connessione insieme, uno leggerebbe la risposta destinata all'altro e il dialogo si sfaserebbe per sempre. Tenere il lucchetto per l'intero scambio, dall'invio alla lettura della risposta, lo impedisce.

Per non restare bloccati in eterno, le letture hanno un tempo massimo di 10 secondi: se l'aggregatore non risponde, il nodo lo considera irraggiungibile. Fa eccezione la RESOLVE, che non ha limite di tempo: può rimanere legittimamente in attesa del token anche a lungo, le specifiche chiedono che una richiesta in attesa non termini con un errore (vedi `AggregatorLink.java`, righe 119-131).

5.3 Come si chiudono in modo ordinato

Il quit di un nodo invia prima DISCONNECT, come chiedono le specifiche, e poi termina. L'aggregatore segna il nodo come inattivo: le sue rilevazioni restano nella tabella ma non vengono più proposte per i download.

Il quit dell'aggregatore avviene in tre passi, in quest'ordine (vedi `Aggregator.java`, metodo `arresta()`, riga 108):

1. chiude il `ServerSocket`, così non entrano nuove connessioni;
2. chiude le connessioni di tutti i nodi collegati, lavorando su una copia della lista presa sotto il lucchetto;
3. termina il processo con `System.exit(0)`. Questo passo è necessario: chiudere un socket sveglia i thread che stanno leggendo, ma non quelli addormentati in attesa di un token, che altrimenti terrebbero vivo il programma.

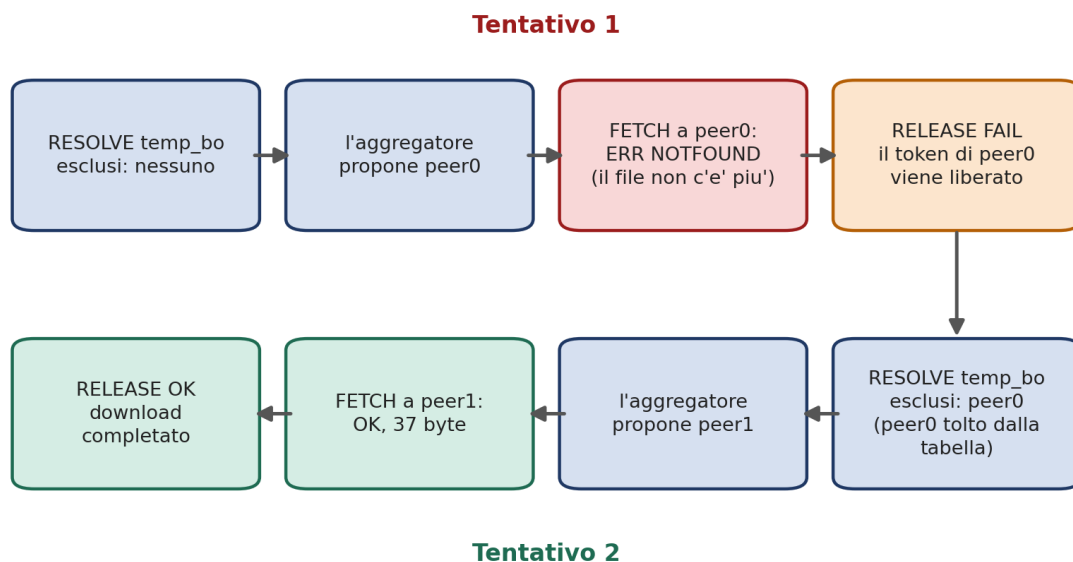
5.4 Cosa succede in caso di interruzioni anomale

Situazione	Comportamento
Un nodo cade (crash, finestra chiusa)	Il suo <code>NodeHandler</code> rileva la fine della connessione ed esegue la pulizia: libera i token del nodo, scrive nel log i download interrotti, lo segna come inattivo.
Un nodo cade mentre tiene un token	La pulizia libera il token e sveglia chi lo aspettava, che viene servito normalmente.
Un nodo cade mentre aspetta un token	Il suo <code>NodeHandler</code> è addormentato e non si accorge di nulla. Quando il token si libera si sveglia, lo prende per un nodo ormai scomparso, e se ne accorge solo scrivendo la risposta. Per questo la pulizia sta in un blocco <code>finally</code> che viene eseguito in ogni caso (vedi <code>NodeHandler.java</code> , metodo <code>pulizia()</code> , riga 328).
Il nodo sorgente cade a metà trasferimento	Il download fallisce e il nodo che scaricava riprova su un altro possessore (paragrafo 5.5).
L'aggregatore cade	I nodi se ne accorgono al comando successivo e mostrano "aggregatore non più raggiungibile". Non è prevista una riconnessione automatica.

Situazione	Comportamento
Aggregatore irraggiungibile all'avvio del nodo	Messaggio d'errore e uscita.
Cartella dati inesistente	Messaggio d'errore e uscita, prima ancora di collegarsi.
Messaggio malformato	Risposta ERR_BADREQUEST; la connessione resta aperta. Una riga oltre gli 8 KB invece chiude la connessione, perché non si sa più dove finisce il messaggio.

5.5 Il download robusto

La tabella dell'aggregatore può non corrispondere alla realtà: un file può essere stato cancellato dal disco senza avvisare nessuno, oppure un nodo può essere caduto. Per questo le specifiche chiedono un download robusto, che proviamo con la Figura 5.



Se il nodo proposto non ha più la rilevazione, il DownloadManager rilascia il token di quel nodo, lo aggiunge a una lista di **esclusi** e chiede all'aggregatore un altro candidato, mandando la lista degli esclusi. L'aggregatore toglie dalla tabella i nodi esclusi e ne propone un altro. Si continua finché il download riesce o finché non ci sono più candidati: in quel caso il nodo risponde che la rilevazione non è disponibile sulla rete.

Il token viene rilasciato **prima** di chiedere un nuovo candidato: altrimenti un solo download terrebbe occupati tutti i nodi che ha già scartato. La lista degli esclusi garantisce che il ciclo finisca, perché ogni tentativo fallito toglie un candidato.

Esempio reale. Abbiamo cancellato temp_bo.txt dal disco del nodo A senza avvisare l'aggregatore; il nodo C, chiedendo temp_bo, ha stampato:

```
> download temp_bo  
Tentativo su peer0 fallito (risorsa non disponibile), riprovo...  
Scaricata 'temp_bo' da peer1 (37 byte).
```

6. Problemi incontrati, soluzioni e alternative

Questa sezione raccoglie i problemi più significativi affrontati durante lo sviluppo, come li abbiamo risolti e quali alternative abbiamo scartato.

Il significato del token

In un primo momento avevamo interpretato il token come un semplice numero di pratica, che identificava un download senza limitare nulla. Poiché la specifica era ambigua, abbiamo chiesto ai tutor, che hanno chiarito che il token regola l'accesso a un nodo. Il cambiamento ha avuto tre conseguenze: TokenManager è diventato un punto di attesa; la RESOLVE può rispondere dopo molto tempo, e quindi per lei abbiamo tolto il limite di tempo; il token va rilasciato prima di ogni nuovo tentativo.

La registrazione che poteva bloccare l'aggregatore

In una prima versione l'aggregatore bloccava la tabella mentre leggeva le righe della REGISTER. Un nodo che avesse inviato l'intestazione e poi più nulla avrebbe fermato l'intero aggregatore. Abbiamo risolto leggendo prima tutte le righe e aggiornando la tabella solo alla fine, in un'unica operazione.

Il contenuto delle rilevazioni corrotto

Per leggere righe da un socket il corso mostra la classe BufferedReader. Nel nostro caso però non va bene: per efficienza legge in anticipo un blocco di dati, e dopo la riga LENGTH si porterebbe via anche i byte della rilevazione, che non arriverebbero più a chi li deve leggere. Per questo Protocol.readLine() legge un byte alla volta, fermandosi esattamente alla fine della riga (vedi Protocol.java, riga 90). BufferedReader è usato solo per leggere la tastiera.

I file che arrivano a metà

Una lettura da socket può restituire meno byte di quelli richiesti, anche se il mittente li ha inviati tutti, perché i dati viaggiano a pezzi. Protocol.readFully() continua a leggere finché non li ha ricevuti tutti (riga 157). Senza questo ciclo i file risulterebbero troncati proprio quando la rete è lenta.

Rilasciare il token di un altro nodo

Durante i test abbiamo scoperto che un nodo poteva inviare la RELEASE di un token che non era suo, liberando un nodo mentre un altro lo stava ancora usando. Ora l'aggregatore controlla che il token appartenga a chi lo rilascia, e in caso contrario risponde ERR_BADTOKEN senza fare nulla (vedi NodeHandler.java, riga 280).

Il file nascosto di macOS

macOS crea un file nascosto .DS_Store in ogni cartella aperta dal Finder. Il nodo lo considerava una rilevazione, e la annunciava a tutta la rete con un nome senza senso. Ora LocalStore considera solo i file che finiscono in .txt (vedi LocalStore.java, riga 43).

Dati e presentazione

In una prima versione la tabella restituiva le righe già pronte per lo schermo (temp_bo: peer0, peer1), che poi finivano così anche sul socket, rompendo il formato del protocollo. Abbiamo separato i compiti: la tabella restituisce i dati nel formato del protocollo (temp_bo peer0 peer1), e sono le console a trasformarli nel formato richiesto dalle specifiche.

6.1 Scelte adottate e alternative scartate

Scelta adottata	Alternativa scartata	Motivo
synchronized, wait(), notifyAll()	Classi di java.util.concurrent	Coerenza e codice più semplice
Mutua esclusione totale sulla tabella	Letture in parallelo (ReadWriteLock)	Le specifiche chiedono che anche le letture attendano durante un aggiornamento
Connessione persistente con l'aggregatore	Una connessione nuova per ogni comando	Non si potrebbe distinguere un quit da un crash
Protocollo di testo su socket	RMI o invio di oggetti Java	Il testo si prova a mano e si legge nei log
Lunghezza dichiarata con LENGTH	Una parola che segnala la fine	La parola potrebbe comparire dentro una rilevazione
Si aspetta il nodo scelto	Preferire un nodo con il token libero	Più efficiente, ma intreccia tabella e token; resta un possibile miglioramento
Nessun limite di tempo per la RESOLVE	Stesso limite di 10 secondi	L'attesa del token è legittima e non deve finire in errore
Terzo argomento opzionale per la cartella	Una cartella fissa	Permette più nodi sulla stessa macchina

7. Suddivisione del lavoro

Il lavoro è stato diviso in quattro aree, una per componente. Ciascuno è responsabile della propria area e ne conosce ogni scelta; le parti di confine sono state progettate insieme.

Area	Componente	File	Responsabilità
A	Irene Benevieri	Protocol, AggregatorState, TokenManager	Protocollo; tabella delle rilevazioni (Meccanismo 1); token (Meccanismo 3)
B	Pierfrancesco Musolesi	Aggregator, NodeHandler, AggregatorConsole, DownloadLog	Connessioni e thread dell'aggregatore; esecuzione dei comandi; pulizia dei token; console e log
C	Bianca Garagnani	Client, ClientConsole, AggregatorLink, LocalStore	Avvio del nodo; comandi e formati di output; connessione verso l'aggregatore; archivio locale
D	Francesco Battistini	PeerServer, PeerHandler, PeerAccessLock, DownloadManager	Comunicazione fra nodi; servizio una richiesta alla volta (Meccanismo 2); download robusto

Il lavoro si è svolto in quattro fasi:

1. **Progettazione.** Abbiamo scritto insieme un piano con l'architettura, il protocollo, le risorse condivise e la divisione in aree. Protocol.java, essendo il contratto fra le due applicazioni, è stato definito prima di tutto il resto: finché non era fissato, nessuno poteva lavorare da solo senza rischiare di scrivere una parte incompatibile con quella degli altri.
2. **Sviluppo in parallelo.** Ciascuno ha sviluppato la propria area. L'aggregatore è stato provato da solo, simulando i nodi con nc, prima che il nodo sensore esistesse.
3. **Integrazione e test.** Abbiamo compilato tutti i file insieme, corretto le incompatibilità fra le parti e provato il sistema completo, anche su due computer collegati alla stessa rete.
4. **Documentazione.** La documentazione è stata scritta in gruppo, al termine di ogni macrocategoria.

8. Strumenti utilizzati

Scopo	Strumenti
Sviluppo	Java (JDK 25.0.3), compilazione da terminale con <code>javac</code> , ambiente di sviluppo VS Code e IntelliJ IDEA.
Condivisione del codice	Repository privato su GitHub, gestito con GitHub Desktop per commit, push e pull. Il file <code>.gitignore</code> esclude i file compilati (<code>bin/</code>) e i file di sistema di macOS. I commit sono organizzati per area.
Comunicazione	Gruppo WhatsApp e incontri in presenza
Test	<code>nc</code> (netcat) per provare l'aggregatore a mano; prove finali con più nodi su uno e su due computer.

9. Compilazione e uso, con esempi

9.1 Requisiti

Serve soltanto il Java Development Kit (versione usata dal gruppo: 25.0.3). Non ci sono librerie esterne né strumenti di build.

9.2 Compilazione

Dalla cartella del progetto:

```
javac -d bin src/*.java
```

`src/*.java` passa al compilatore tutti i sorgenti insieme, necessario perché le classi si richiamano a vicenda; `-d bin` scrive i file compilati nella cartella `bin/`, separati dai sorgenti.

9.3 Avvio

Prima l'aggregatore, indicando la porta:

```
java -cp bin Aggregator 9000
Aggregatore in ascolto sulla porta 9000.
```

Poi i nodi sensore, ciascuno in un proprio terminale, con l'indirizzo e la porta dell'aggregatore. Il terzo argomento, facoltativo, è la cartella con le rilevazioni del nodo:

```
java -cp bin Client 127.0.0.1 9000 data/nodeA
Registrato come peer0, in ascolto sulla porta 50231.
```

Senza terzo argomento, cioè con il comando esattamente come nelle specifiche (`java -cp bin Client 127.0.0.1 9000`), il nodo usa la cartella `data/default`, creandola vuota se non

esiste. Una cartella indicata esplicitamente e inesistente produce invece un errore. Gli errori di avvio hanno questa forma:

```
Errore: aggregatore non raggiungibile su 127.0.0.1:9000
Errore: cartella dati non accessibile: data/sbagliata
```

Su più computer. I nodi si collegano con l'indirizzo IP del computer che esegue l'aggregatore, **anche i nodi che girano su quello stesso computer.** Se un nodo si collegasse con 127.0.0.1, l'aggregatore memorizzerebbe quell'indirizzo, e un nodo su un altro computer che provasse a scaricare da lì finirebbe per collegarsi a se stesso.

9.4 Comandi del nodo sensore

Negli esempi sono avviati, in quest'ordine, i nodi A (peer0), B (peer1) e C (peer2) con le cartelle di esempio. I comandi sono digitati sul nodo C, se non indicato diversamente.

listdata local – le rilevazioni del nodo

```
> listdata local
Risorse:
- press_rn
```

listdata remote – tutte le rilevazioni della rete

```
> listdata remote
Risorse:
- co2_aula2: peer0
- co2_aula3: peer1
- press_bo: peer0
- press_rn: peer2
- temp_bo: peer0, peer1
```

listdata remote <nome> – chi possiede una rilevazione

```
> listdata remote temp_bo
Risorse:
- temp_bo: peer0, peer1
```

listdata peers – gli altri nodi attivi

```
> listdata peers
Nodi attivi:
- peer0
- peer1
```

add <nome> <contenuto> – aggiungere una rilevazione

Il contenuto può contenere spazi; il nome no.

```
> add umidita_bo 61 per cento
Rilevazione 'umidita_bo' aggiunta e notificata all'aggregatore.
```

remove <nome> – rimuovere una rilevazione

```
> remove umidita_bo
Rilevazione 'umidita_bo' rimossa e notificata all'aggregatore.
```

download <nome> – scaricare dalla rete

L'aggregatore sceglie il nodo sorgente. Gli altri due esiti possibili sono una rilevazione già posseduta e una rilevazione che nessuno ha:

```
> download press_bo
Scaricata 'press_bo' da peer0 (37 byte).
> download press_bo
Rilevazione 'press_bo' gia' presente in locale.
> download inesistente
Rilevazione 'inesistente' non disponibile sulla rete.
```

download <peer> <nome> – scaricare da un nodo scelto

Digitato sul nodo B:

```
> download peer2 press_rn
Scaricata 'press_rn' da peer2 (36 byte).
> download peer2 co2_aula3
Rilevazione 'co2_aula3' non disponibile su peer2.
```

help e quit

help elenca i comandi; quit avvisa l'aggregatore e chiude il nodo:

```
> quit
Nodo arrestato.
```

9.5 Comandi dell'aggregatore**listdata – le rilevazioni della rete**

Stesso formato di listdata remote sul nodo:

```
> listdata
Risorse:
- co2_aula2: peer0
- co2_aula3: peer1
- press_bo: peer0, peer2
- press_rn: peer2
- temp_bo: peer0, peer1
```

log – il registro dei download

```
> log
Risorse scaricate:
- 13:11:23 press_bo da: peer0 a: peer2 esito: OK
- 13:11:26 inesistente da: - a: peer2 esito: NON_DISPONIBILE
- 13:11:44 temp_bo da: peer0 a: peer2 esito: FALLITO
- 13:11:44 temp_bo da: peer1 a: peer2 esito: OK
```

Rispetto all'esempio delle specifiche abbiamo aggiunto due cose. L'**esito** di ogni richiesta, che il testo delle specifiche richiede ("se sono andate a buon fine") anche se l'esempio non lo mostra. E i **secondi** nell'orario, perché i tentativi di uno stesso download avvengono quasi sempre nello stesso minuto e altrimenti sarebbero indistinguibili.

help e quit

help elenca i comandi; quit chiude tutte le connessioni e arresta l'aggregatore:

```
> quit
Aggregatore arrestato.
```

I nodi ancora collegati se ne accorgono al comando successivo:

```
> listdata remote
Errore: aggregatore non piu' raggiungibile.
```

9.6 Estensioni implementate

Alcune funzionalità descritte nelle specifiche non avevano un comando corrispondente. Le abbiamo aggiunte dopo averne verificato la coerenza con i tutor, mantenendo invariati tutti i comandi richiesti:

- `listdata peers`: la lista degli altri nodi attivi;
- `listdata remote <nome>`: i possessori di una singola rilevazione;
- `download <peer> <nome>`: il download da un nodo scelto dall'utente;
- `remove <nome>`: la rimozione di una rilevazione, che le specifiche citano fra le cause di disallineamento della tabella.

9.7 Rendere visibili le attese durante una prova

I trasferimenti durano pochi millisecondi, quindi le attese dei Meccanismi 2 e 3 non si vedrebbero. Per mostrarle si può avviare un nodo con un ritardo artificiale, in millisecondi, applicato mentre serve una richiesta (vedi `PeerHandler.java`, riga 18). Il ritardo è spento se non viene indicato:

```
java -Dlabso.slow=3000 -cp bin Client 127.0.0.1 9000 data/nodeA
```

Con il nodo A avviato così, due nodi che scaricano da A nello stesso momento mostrano chiaramente il Meccanismo 3 (paragrafo 4.3), e due richieste dirette ad A il Meccanismo 2 (paragrafo 4.2).